

Database Systems Lab — Week 2 (Relational Model, Keys, ER, Normalization)

Task 1.1 (superkeys / candidate keys for Employee and Registration) was done already, so it's not repeated here. Below is everything from Task 1.2 to the end.

Task 1.2 — Foreign Key Design

Tables: Student(StudentID, Name, Email, Major, AdvisorID), Professor(ProfID, Name, Department, Salary), Course(CourseID, Title, Credits, DepartmentCode), Department(DeptCode, DeptName, Budget, ChairID), Enrollment(StudentID, CourseID, Semester, Grade).

FK column	In table	Points to	Why
AdvisorID	Student	Professor.ProfID	every student has one advisor, who is a professor
Department	Professor	Department.DeptCode	professor belongs to one department
DepartmentCode	Course	Department.DeptCode	course is owned by one department
ChairID	Department	Professor.ProfID	department has one chair, and chair is a professor
StudentID	Enrollment	Student.StudentID	enrollment always belongs to one student
CourseID	Enrollment	Course.CourseID	enrollment always belongs to one course

Tricky part: Department.ChairID points to Professor, and Professor.Department points back to Department. That's a circular reference — you can't insert one without the other already existing. Easiest fix: let ChairID be NULL at first, insert both rows, then update ChairID afterwards.

Part 2 — ER Diagrams

Task 2.1 — Hospital Management System

Entities:

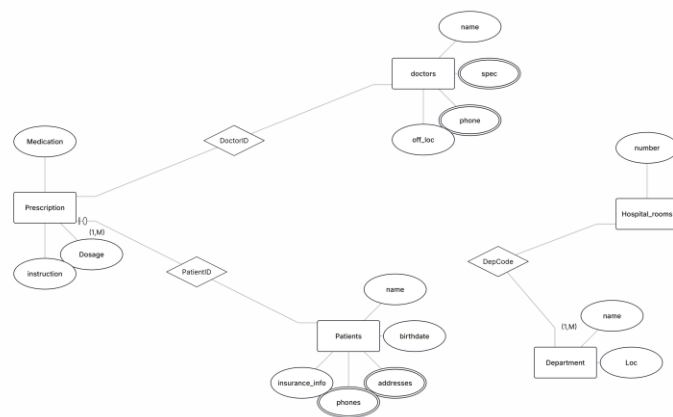
Entity	Strong / weak	Key
Patient	Strong	PatientID
Doctor	Strong	DoctorID
Department	Strong	DeptCode
Room	Weak — same room number repeats across departments, needs department to be unique	RoomNumber + DeptCode
Medication	Strong	MedicationID

Appointment and Prescription aren't separate entities on their own — they're relationships with their own attributes (Appointment has date/time/purpose/notes, Prescription has dosage/instructions), since that's exactly what they are: a link between people that carries extra info.

Attributes: Patient's Name and Address are composite (made of smaller parts), PhoneNumbers is multi-valued (a patient can have more than one), and Age would be a derived attribute if added (calculated from BirthDate). Doctor's Specializations is multi-valued too.

Relationships & cardinality: Patient–Appointment–Doctor is M:N. Doctor–Department is N:1. Department–Room is 1:N and identifying (Room is weak). Doctor–Patient–Medication (Prescription) is a three-way relationship, drawn as one diamond touching all three entities instead of two separate M:N relationships.

Figure 1. Hospital ER diagram (Patient, Doctor, Department, Room, Medication, Appointment, Prescription)

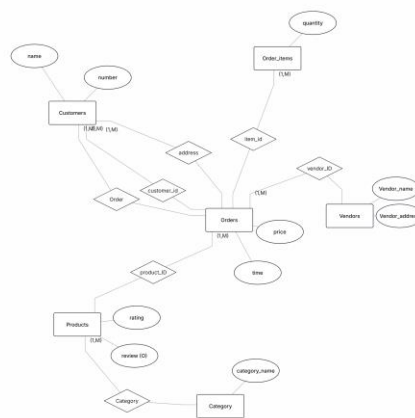


Task 2.2 — E-commerce Platform

Weak entity: OrderItem. It doesn't mean anything by itself — a line item only exists because some Order exists, and its key is OrderID + ProductID together, borrowing OrderID from the order it belongs to. Delete the order, the line items go with it.

M:N with attributes: Order and Product — one order has many products, one product appears in many orders, and this needs Quantity and PriceAtOrder (price at the moment of purchase). Same idea for Customer and Product through Review — needs Rating, Comment and Date.

Figure 2. E-commerce ER diagram (Customer, Address, Order, OrderItem, Product, Category, Vendor, Inventory, Review)



Part 4 — Normalization

Task 4.1 — StudentProject table

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

Functional dependencies:

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

What's wrong with it: student's name and major get repeated on every project row they work on. Same with project title/type/supervisor — repeated for every student on that project. That's the redundancy.

- **Update:** student changes major → have to fix every row with that StudentID, easy to miss one and get inconsistent data.
- **Insert:** can't add a new project until some student is already assigned to it (and vice versa).
- **Delete:** remove the last student on a project and the project's title/type/supervisor info disappears too.

1NF: already fine — no repeating groups, every cell holds one value.

2NF: primary key is (StudentID, ProjectID). But StudentID alone already gives StudentName/StudentMajor, and ProjectID alone already gives ProjectTitle/ProjectType/SupervisorID — partial dependency, breaks 2NF.

- Student(StudentID, StudentName, StudentMajor)
- Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)
- Assignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

3NF: inside the new Project table, SupervisorID → SupervisorName/SupervisorDept, but SupervisorID isn't the key of that table — transitive dependency, split it out too.

- **Student**(StudentID, StudentName, StudentMajor)
- **Supervisor**(SupervisorID, SupervisorName, SupervisorDept)
- **Project**(ProjectID, ProjectTitle, ProjectType, SupervisorID)
- **Assignment**(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

Task 4.2 — CourseSchedule table

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

Primary key (the trick): there's no Section attribute here, unlike Task 1.1's Registration table. The rules say each course is taught at one time by one instructor in one room — so CourseID already means one fixed offering, not a course with several sections. So a row is just "this student takes this course", key is {StudentID, CourseID}.

Functional dependencies:

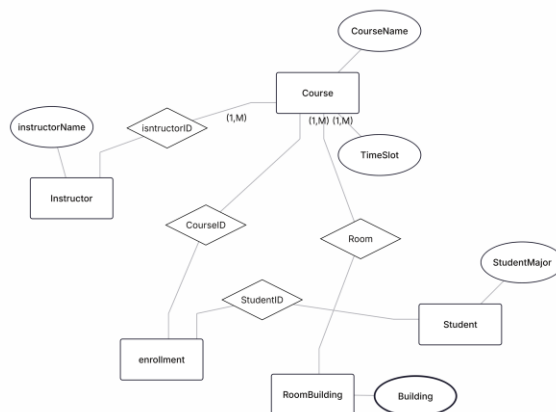
- StudentID → StudentMajor
- CourseID → CourseName, InstructorID, TimeSlot, Room
- InstructorID → InstructorName
- Room → Building (rooms are unique campus-wide, Room alone is enough — don't need TimeSlot for this one)

BCNF check: for BCNF every left side of an FD has to be a superkey. The real superkey here is {StudentID, CourseID}, but StudentID, CourseID, InstructorID and Room are all determinants on their own and none of them is the full key — so all four FDs above break BCNF.

Decomposition (split off each violation one at a time):

- **Student**(StudentID, StudentMajor)
- **Instructor**(InstructorID, InstructorName)
- **RoomBuilding**(Room, Building)
- **Course**(CourseID, CourseName, InstructorID, TimeSlot, Room)
- **Enrollment**(StudentID, CourseID)

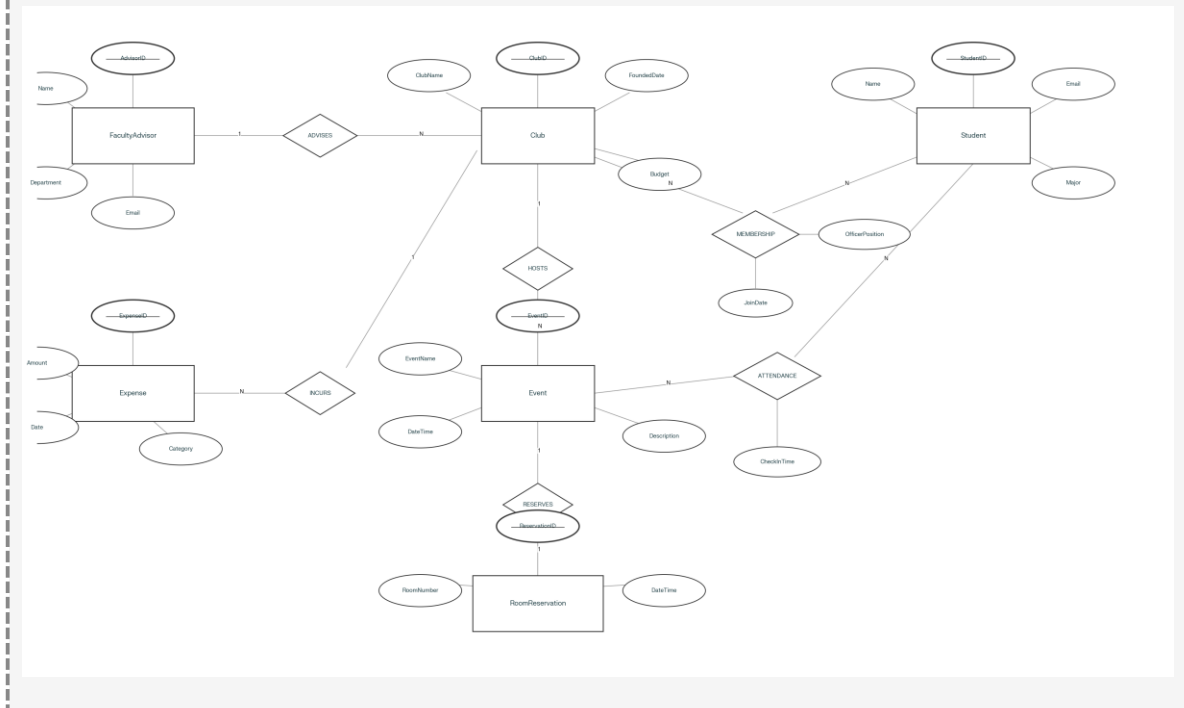
Figure 3. BCNF decomposition of CourseSchedule



Any data lost? No — joining everything back gives the exact same table. What you do lose is the automatic guarantee that "one instructor can't teach two classes at the same time" or "one room can't host two classes at the same time" — now that needs a separate unique constraint on (InstructorID, TimeSlot) and (Room, TimeSlot) in the Course table.

Part 5 — Student Clubs & Organizations

Figure 4. ER diagram for the clubs system (Student, Club, FacultyAdvisor, Membership, Event, Attendance, RoomReservation, Expense)



Relational schema:

- **Student**(StudentID, Name, Email, Major)
- **FacultyAdvisor**(AdvisorID, Name, Department, Email)
- **Club**(ClubID, ClubName, FoundedDate, Budget, AdvisorID FK)
- **Membership**(StudentID FK, ClubID FK, JoinDate, OfficerPosition)
- **Event**(EventID, EventName, DateTime, Description, ClubID FK)
- **Attendance**(StudentID FK, EventID FK, CheckInTime)
- **RoomReservation**(ReservationID, RoomNumber, DateTime, EventID FK)
- **Expense**(ExpenseID, ClubID FK, Amount, Date, Category)

Design choice I made: officer position (president/treasurer/secretary) could either be just a field on Membership, or a whole separate table tracking when someone became/stopped being an officer. Went with the simple field on Membership — good enough if you only care who's the officer right now, not the full history.

Example queries:

- Find all students who are officers in the Computer Science Club.
- List all events scheduled for next week with their room reservations.
- Total up each club's expenses this semester and check it against their budget.